

CO2-ASSESSMENT TOOL-3

Name: SIVASUDHAN N

Reg.no: 192421017

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 29-07-2026

Q34

Title

Debugging Binary Search Code (Incorrect Mid Recalculation Logic)

Problem Statement

The given Python program is intended to perform Binary Search on a sorted list. However, the implementation contains a major issue because the search boundaries (low and high) are updated incorrectly. Due to this mistake, the algorithm searches in the wrong direction and may fail to find an element even if it exists.

The objective is to debug the code by correcting the boundary updates, ensuring that the search interval shrinks correctly, improving code clarity, handling edge cases, and maintaining efficient Binary Search functionality.

Original Code

```
1 def binary_search(arr, key):
2     low, high = 0, len(arr)-1
3
4     while low <= high:
5         mid = (low + high) // 2
6
7         if arr[mid] == key:
8             return mid
9
10        elif arr[mid] > key:
11            low = mid + 1      # ERROR
12
13        else:
14            high = mid - 1    # ERROR
15
16    return -1
```

Problem Analysis

Issue 1: Incorrect Boundary Update

- When $\text{arr}[\text{mid}] > \text{key}$, the search should move to the left half.
- The program incorrectly updates $\text{low} = \text{mid} + 1$.

Issue 2: Incorrect Search Direction

- When $\text{arr}[\text{mid}] < \text{key}$, the search should move to the right half.
- The program incorrectly updates $\text{high} = \text{mid} - 1$.

Issue 3: Wrong Search Interval

- Because the boundaries are reversed, valid portions of the array are discarded.
- The element may never be found even though it exists.

Corrected Algorithm

1. Accept the sorted input list and search key.

2. Initialize `low = 0` and `high = len(arr)-1`.
3. Calculate the middle index.
4. Compare the middle element with the search key.
5. If both are equal, return the index.
6. If the key is smaller, update `high = mid - 1`.
7. If the key is larger, update `low = mid + 1`.
8. Repeat until the element is found or the search interval becomes empty.
9. Return -1 if the element is not present.

Python Program (Source Code)

```
1 def binary_search(arr, key):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6
7         mid = (low + high) // 2
8
9         if arr[mid] == key:
10            return mid
11
12        elif arr[mid] > key:
13            high = mid - 1
14
15        else:
16            low = mid + 1
17
18    return -1
19
20
21 numbers = [10, 20, 30, 40, 50, 60, 70]
22
23 key = 50
```

OUTPUT

```
Element found at index: 4
```

Example Output

Element found at index: 4

Explanation

- The corrected program initializes the search boundaries properly.
- It calculates the middle element during each iteration.
- If the key matches the middle element, its index is returned.
- If the key is smaller than the middle element, the algorithm searches the left half.
- If the key is larger than the middle element, the algorithm searches the right half.
- If the search interval becomes empty, the function returns -1.

This correction ensures that Binary Search works correctly and efficiently on sorted data.

Time Complexity Analysis

Operation	Complexity
Initialize Variables	$O(1)$
Binary Search	$O(\log n)$
Overall Time Complexity $O(\log n)$	
Space Complexity	$O(1)$

Best Case

- The key is found in the first comparison.
- Time Complexity: $O(1)$

Average Case

- The search interval is repeatedly divided into two halves.
- Time Complexity: $O(\log n)$

Worst Case

- The element is at the last possible position or is not present.
- Time Complexity: $O(\log n)$

Advantages

- Correctly updates the search boundaries.
- Prevents incorrect search direction.
- Efficient for large sorted datasets.
- Easy to understand and maintain.
- Uses logarithmic time complexity.
- Handles edge cases correctly.

Analysis/Documentation Screenshot

```
1 # Analysis of Binary Search
2
3 arr = [10, 20, 30, 40, 50, 60, 70]
4
5 print("Array:", arr)
6 print("Number of Elements:", len(arr))
7
8 print("\nTime Complexity Analysis")
9 print("Best Case      : O(1)")
10 print("Average Case   : O(log n)")
11 print("Worst Case     : O(log n)")
12 print("Space Complexity : O(1)")
```

OUTPUT

```
Array: [10, 20, 30, 40, 50, 60, 70]
Number of Elements: 7

Time Complexity Analysis
Best Case      : O(1)
Average Case   : O(log n)
Worst Case     : O(log n)
Space Complexity : O(1)
```

Conclusion

The original Binary Search implementation failed because the updates for low and high were reversed, causing the algorithm to discard the correct search region. The corrected version updates the boundaries properly, searches in the correct direction, and successfully finds the required element. The algorithm achieves $O(\log n)$ time complexity and $O(1)$ space complexity, making it an efficient and reliable searching technique for sorted data.